1. Introduction

This project implements a simple Library Management System using C# and .NET.

The system allows users to:

- Add new books
- Register borrowers
- Borrow books
- Return books
- View all books and borrowers

Unit testing is implemented using **xUnit** to ensure all functionalities work correctly.

---

2. Technologies Used

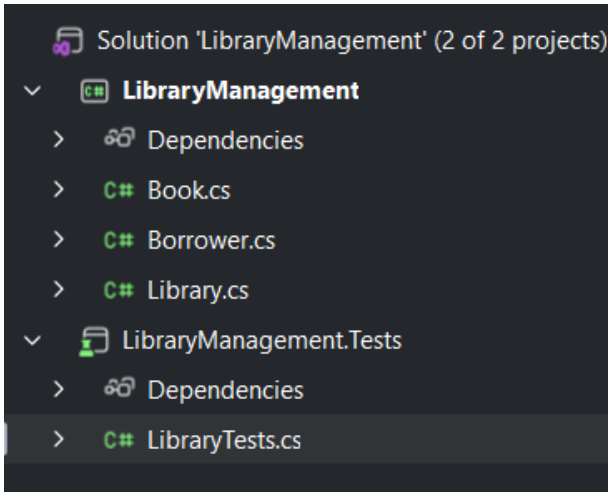
- C#
- .NET Class Library
- xUnit Testing Framework
- Visual Studio

---

3. Project Structure

The solution contains two projects:

1. **LibraryManagement** (Class Library)
2. **LibraryManagement.Tests** (xUnit Test Project)



4. Class Implementations

4.1 Book Class

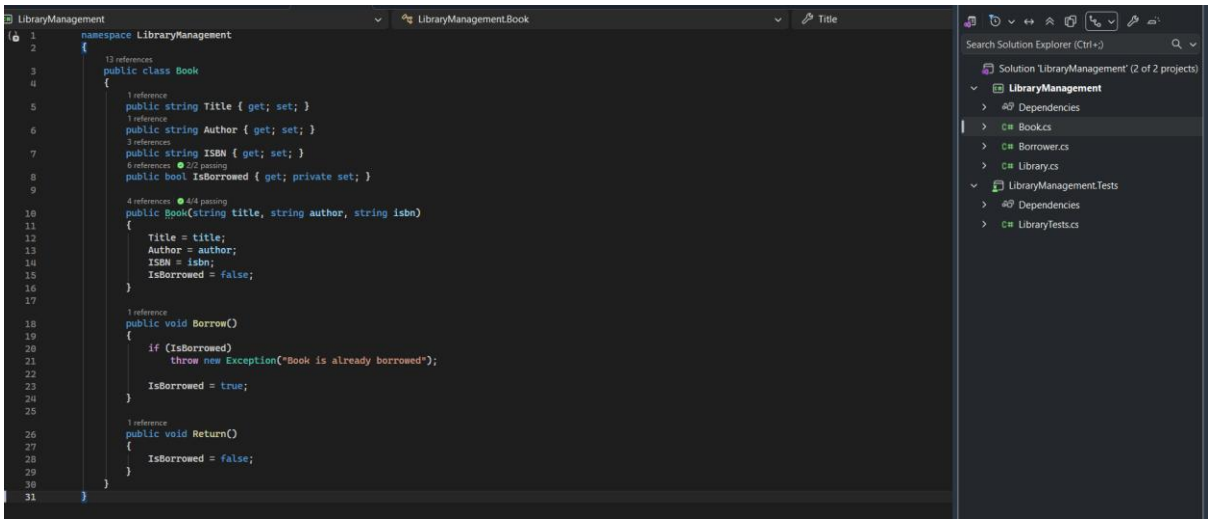
The Book class represents a book in the library.

Properties:

- Title
- Author
- ISBN
- IsBorrowed

Methods:

- Borrow() – Marks the book as borrowed
- Return() – Marks the book as available



CODE :

```
namespace LibraryManagement
{
    public class Book
    {
        public string Title { get; set; }
        public string Author { get; set; }
        public string ISBN { get; set; }
        public bool IsBorrowed { get; private set; }

        public Book(string title, string author, string isbn)
        {
            Title = title;
            Author = author;
            ISBN = isbn;
            IsBorrowed = false;
        }

        public void Borrow()
        {
            if (IsBorrowed)
                throw new Exception("Book is already borrowed");

            IsBorrowed = true;
        }

        public void Return()
```

```
        {
            IsBorrowed = false;
        }
    }
}
```

4.2 Borrower Class

The Borrower class represents a registered library member.

Properties:

- Name
- LibraryCardNumber
- BorrowedBooks (List of books)

Methods:

- BorrowBook(Book book)
- ReturnBook(Book book)

Code :-

```
using System.Collections.Generic;

namespace LibraryManagement
{
    public class Borrower
    {
        public string Name { get; set; }
        public string LibraryCardNumber { get; set; }
        public List<Book> BorrowedBooks { get; private set; }

        public Borrower(string name, string cardNumber)
        {
            Name = name;
            LibraryCardNumber = cardNumber;
            BorrowedBooks = new List<Book>();
        }

        public void BorrowBook(Book book)
        {
            book.Borrow();
            BorrowedBooks.Add(book);
        }
```

```
        public void ReturnBook(Book book)
        {
            book.Return();
            BorrowedBooks.Remove(book);
        }
    }
}
```

4.3 Library Class

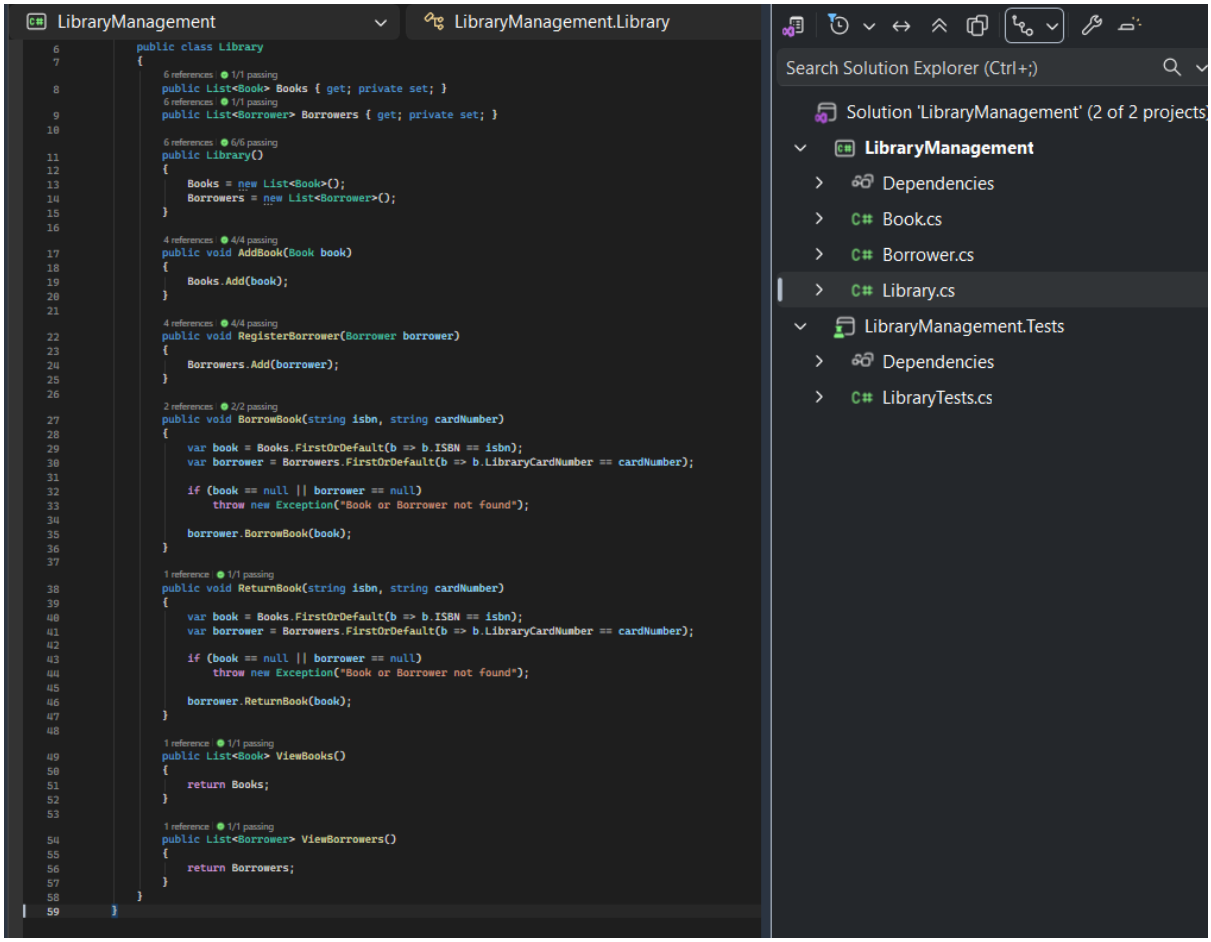
The Library class manages all books and borrowers.

Properties:

- Books (List of Book)
- Borrowers (List of Borrower)

Methods:

- AddBook(Book book)
- RegisterBorrower(Borrower borrower)
- BorrowBook(string isbn, string libraryCardNumber)
- ReturnBook(string isbn, string libraryCardNumber)
- ViewBooks()
- ViewBorrowers()



Code :

```
using System.Collections.Generic;
using System.Linq;
```

```
namespace LibraryManagement
{
    public class Library
    {
        public List<Book> Books { get; private set; }
        public List<Borrower> Borrowers { get; private set; }

        public Library()
        {
            Books = new List<Book>();
            Borrowers = new List<Borrower>();
        }

        public void AddBook(Book book)
        {
            Books.Add(book);
        }

        public void RegisterBorrower(Borrower borrower)
        {
            Borrowers.Add(borrower);
        }

        public void BorrowBook(string isbn, string cardNumber)
        {
            var book = Books.FirstOrDefault(b => b.ISBN == isbn);
            var borrower = Borrowers.FirstOrDefault(b => b.LibraryCardNumber ==
cardNumber);

            if (book == null || borrower == null)
                throw new Exception("Book or Borrower not found");

            borrower.BorrowBook(book);
        }

        public void ReturnBook(string isbn, string cardNumber)
        {
            var book = Books.FirstOrDefault(b => b.ISBN == isbn);
            var borrower = Borrowers.FirstOrDefault(b => b.LibraryCardNumber ==
cardNumber);

            if (book == null || borrower == null)
                throw new Exception("Book or Borrower not found");

            borrower.ReturnBook(book);
        }

        public List<Book> ViewBooks()
        {
            return Books;
        }

        public List<Borrower> ViewBorrowers()
        {
            return Borrowers;
        }
    }
}
```

5. Functional Implementation

The system performs the following operations:

1. Adding a Book

Stores book details in the Books list.

2. Registering a Borrower

Stores borrower details in the Borrowers list.

3. Borrowing a Book

- Marks book as borrowed
- Associates book with borrower

4. Returning a Book

- Marks book as available
- Removes book from borrower list

5. Viewing Books and Borrowers

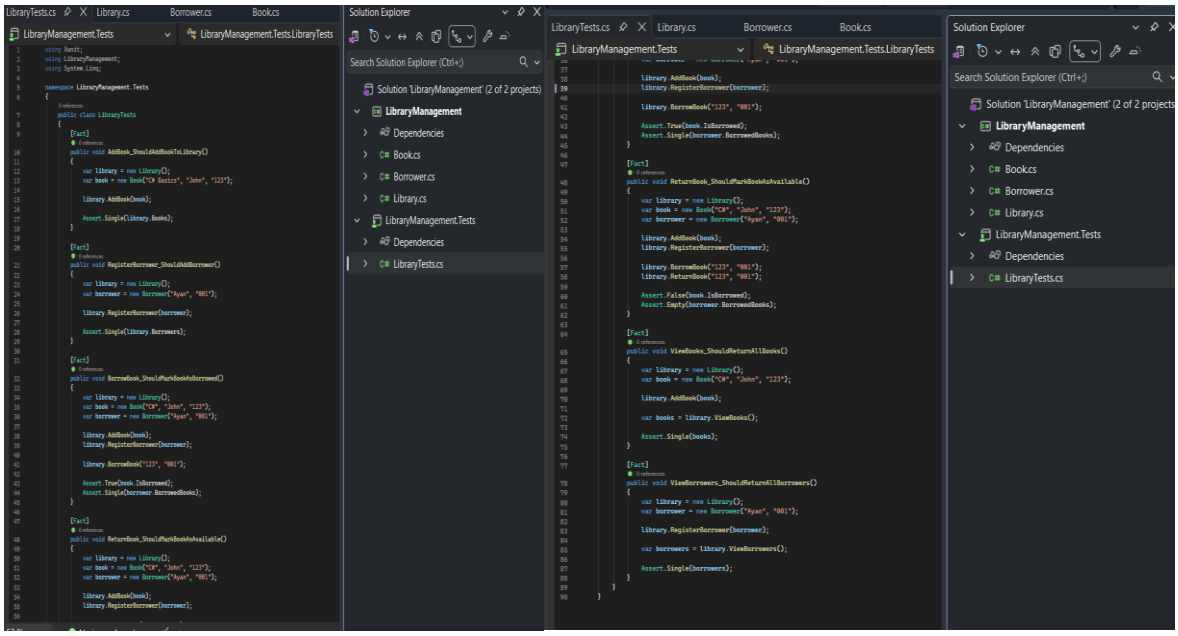
Displays list of all books and registered borrowers.

6. Unit Testing

Unit tests are implemented using **xUnit** framework.

The following test cases are covered:

1. AddBook_ShouldAddBookToLibrary
2. RegisterBorrower_ShouldAddBorrower
3. BorrowBook_ShouldMarkBookAsBorrowed
4. ReturnBook_ShouldMarkBookAsAvailable
5. ViewBooks_ShouldReturnAllBooks
6. ViewBorrowers_ShouldReturnAllBorrowers



Code:

```
using Xunit;
using LibraryManagement;
using System.Linq;
```

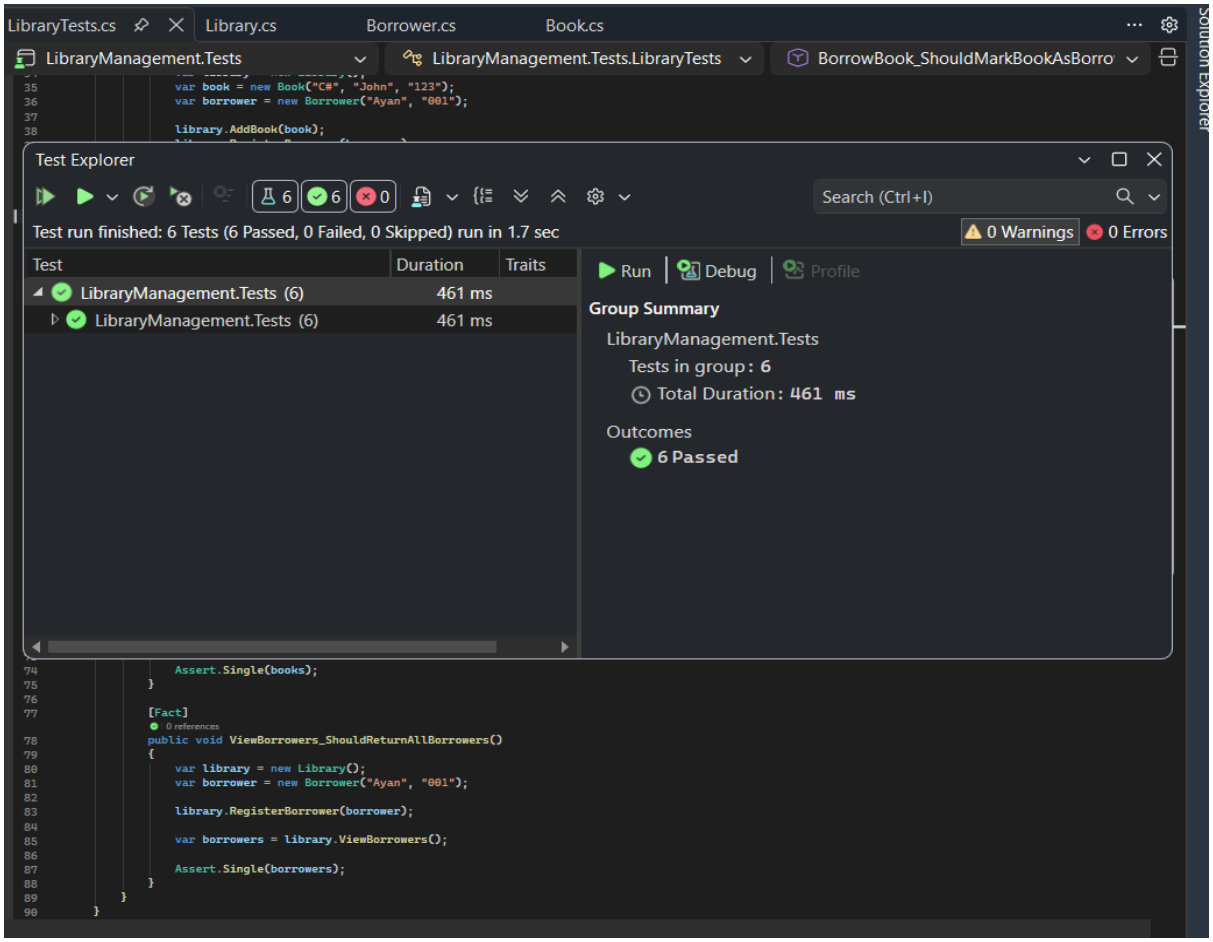
```
namespace LibraryManagement.Tests
{
    public class LibraryTests
    {
        [Fact]
        public void AddBook_ShouldAddBookToLibrary()
        {
            var library = new Library();
            var book = new Book("C# Basics", "John", "123");

            library.AddBook(book);

            Assert.Single(library.Books);
        }

        [Fact]
        public void RegisterBorrower_ShouldAddBorrower()
        {
            var library = new Library();
            var borrower = new Borrower("Ayan", "001");

            library.RegisterBorrower(borrower);

            Assert.Single(library.Borrowers);
        }

        [Fact]
        public void BorrowBook_ShouldMarkBookAsBorrowed()
        {
            var library = new Library();
            var book = new Book("C#", "John", "123");
            var borrower = new Borrower("Ayan", "001");

            library.AddBook(book);
            library.RegisterBorrower(borrower);

            library.BorrowBook("123", "001");

            Assert.True(book.IsBorrowed);
            Assert.Single(borrower.BorrowedBooks);
        }

        [Fact]
        public void ReturnBook_ShouldMarkBookAsAvailable()
        {
            var library = new Library();
            var book = new Book("C#", "John", "123");
            var borrower = new Borrower("Ayan", "001");

            library.AddBook(book);
            library.RegisterBorrower(borrower);

            library.BorrowBook("123", "001");
            library.ReturnBook("123", "001");

            Assert.False(book.IsBorrowed);
            Assert.Empty(borrower.BorrowedBooks);
        }

        [Fact]
        public void ViewBooks_ShouldReturnAllBooks()
        {
            var library = new Library();
            var book = new Book("C#", "John", "123");

            library.AddBook(book);

            var books = library.ViewBooks();

            Assert.Single(books);
```

```
        }

        [Fact]
        public void ViewBorrowers_ShouldReturnAllBorrowers()
        {
            var library = new Library();
            var borrower = new Borrower("Ayan", "001");

            library.RegisterBorrower(borrower);

            var borrowers = library.ViewBorrowers();

            Assert.Single(borrowers);
        }
    }
}
```

---

7. Test Execution

All test cases were executed using Test Explorer in Visual Studio.



All tests passed successfully without errors.

---

8. Exception Handling

The system includes basic exception handling:

- Prevents borrowing an already borrowed book.
- Handles cases where book or borrower is not found.

9. Conclusion

This project demonstrates:

- Object-Oriented Programming concepts
- Proper class design
- List data management
- Unit testing using xUnit
- Clean and structured solution architecture

All requirements mentioned in the assignment have been successfully implemented and tested.